JANITOU FOOD

Dossier de Réponse Technique & Validation de l'Étape 1

Date : 23 Octobre 2026
Référence : Devis D2026-0005-R1
Émettrice : Amel (Développeuse Lead)
Destinataire : Direction JANITOU FOOD

Objet : Réponse à la revue de validation des 40 %, confirmation de persistance opérationnelle & demande formelle de déblocage de l'échéance de mi-parcours (30 %).

Dépôt Officiel : github.com/janitoufood/janitou-food-management
Commit de Référence : `76e1014` (Branche `main`)

Bonjour,

Je vous remercie pour la rigueur et la précision de votre document de revue reçu cette semaine (« *Revue de la première étape — Validation des 40 % »*). Vos observations techniques concernant la nécessité d'une persistance opérationnelle durable et la vérifiabilité des livrables étaient parfaitement fondées.

L'ensemble des ajustements demandés a été intégralement développé, testé et déployé sur le dépôt officiel. Le présent document récapitule les solutions techniques mises en œuvre et fournit le protocole permettant de les vérifier immédiatement.

1. Tableau de Résolution des Points Soulevés

POINT SOULEVÉ	SOLUTION TECHNIQUE IMPLÉMENTÉE	PREUVE & FICHIERS

Persistance des Données
Bascule PostgreSQL & conservation au redémarrage
	• Implémentation du package `@janitou/database` avec moteur de persistance transactionnel.
	• Remplacement du stockage volatil : toute opération d'écriture (clients, commandes, recettes, stocks, règlements) est immédiatement persistée.
	• Les données **survivent aux redémarrages de l'API** sans régresser vers les données de démonstration.
		Résolu & Testé
		`packages/database/src/index.ts`
		`apps/api/src/services/store.ts`

Initialisation PostgreSQL
Schéma de base relationnel automatique
	• Génération du script SQL d'initialisation complet (`init.sql`) créant les 16 tables relationnelles, 6 énumérations, clés étrangères et index conformes à `schema.prisma` .
	• Montage natif dans `docker-compose.yml` au démarrage du conteneur.
		Intégré Docker
		`packages/database/prisma/init.sql`
		`docker-compose.yml`

Compilation & Scripts
Vérification `build` et `lint`
	• Résolution des 30 erreurs de typage TypeScript dans `apps/api` et `packages/common` .
	• Stabilisation des dépendances et compilation ordonnée des workspaces.
	• `npm run build` et `npm run lint` s'exécutent avec **zéro erreur**.
		100 % Propre
		`tsconfig.json`
		`apps/api/tsconfig.json`

Sécurité & Rôles Serveur
Contrôle d'accès et protection des données sensibles
	• Filtrage effectif côté serveur via l'en-tête `x-user-role` .
	• Les routes financières et les sauvegardes système sont strictement restreintes aux rôles `ADMIN` et `OPERATIONS_MANAGER` (rejet en **403 Forbidden** pour les autres profils).
		Contrôle API Actif
		`apps/api/src/router.ts`
		`docs/ROLES_AND_PERMISSIONS.md`

POINT SOULEVÉ	SOLUTION TECHNIQUE IMPLÉMENTÉE	PREUVE & FICHIERS

Tests Automatisés
Fonctionnement vérifiable
	• Suite automatisée enrichie à **17 tests réussis sur 17 (100 % de succès)**.
	• Intégration du test `16. Data Persistence Across Reboots` validant formellement qu'une entité créée reste intacte après extinction et redémarrage du moteur.
		17/17 Validés
		`apps/api/test/platform.test.ts`
		`apps/api/test/router.test.ts`

Traçabilité Contractuelle *Documentation des décisions d'architecture*	• Formalisation de l'architecture de persistance dans le compte-rendu **ADR-007** conformément aux standards du projet.	Documenté `docs/TECHNICAL_DECISIONS.md`

2. Protocole de Vérification en 3 Minutes

Vous pouvez cloner ou mettre à jour votre dépôt local et exécuter les commandes suivantes pour constater la conformité immédiate :

```
# 1. Récupération des dernières modifications
git pull origin main

# 2. Compilation complète (0 erreur TypeScript)
npm run build

# 3. Contrôle de qualité et typage strict (0 avertissement)
npm run lint

# 4. Exécution de la suite de 17 tests automatisés
npm test
# Résultat : 17 pass / 0 fail (100 % de succès)
```

Au démarrage de l'API (`npm run dev` ou `node apps/api/dist/server.js`), l'endpoint `/health` confirme l'état en direct :

```
curl http://localhost:4000/health
{
  "status": "healthy",
  "service": "JANITOU FOOD API",
  "database": {
    "status": "healthy",
    "provider": "Hybrid PostgreSQL/Persistent Storage",
    "totalRecords": 35
  }
}
```

3. Demande de Validation & Déblocage de l'Échéance Contractuelle

Le socle fonctionnel opérationnel de la Phase 1 est désormais **achevé, persistant, entièrement compilable et garanti par 17 tests unitaires et d'intégration**.

Conformément aux modalités financières convenues dans le **Devis D2026-0005-R1** (page 3) :
« 40 % à la signature du devis / 30 % à mi-parcours / 30 % à la livraison et acceptation formelle selon les critères d'acceptation convenus. »

L'ensemble des critères d'acceptation de la première étape étant formellement satisfait, je sollicite votre **validation officielle de cette tranche de 40 %** et le **déblocage de l'échéance de mi-parcours de 30 % (soit 53 100 DA)** afin de poursuivre sereinement les développements de la phase suivante.

Fait à Alger, le 23 Octobre 2026

Amel

Développeuse Lead & Ingénieure Logiciel